1 - Data Addressing Modes

MOV Ax, Bx

opcode → Destination, Source

Segment : off set

CS : IP

DS : SI / DI / BX / 8-bit num / 16-bit num

SS : SP / BP

label , MOV Ax, Bx ← command

* label cannot start with number
* label kyma sebebimiz, eğer instructiona geri dönmek istiyorsak label'i kullanıyoruz.
* command always begin with semicolon (;)

a) Register Addressing ✓

8 bit register : AH, AL, BH, BL, CH, CL, DH, DL

16 bit register : AX, BX, CX, DX, SP, BP, SI, DI

* never mix an 8 bit with a 16 bit register, an 8bit or a 16 bit register with a 32 bit register.

Ex :

MOV AL, BH ✓
MOV BP, SP ✓
MOV CH, CL ✓
MOV CS, DS ✗ → code segment değiştirilemez.
MOV DS, SC ✓
MOV AX, 0A278H ✓ → harf ile başlayamadığı için 0 atıyoruz.
MOV CL, 27H ✓
MOV CH, 0A278H ✗ → CH sabit, değişmez.
MOV 78H, CL ✗ → Destination port cannot be constant
MOV CH, 'x' ✓
MOV CX, 'x' ✗ → CX : 2 bit 'x' : 1bit

* Content of the source part does'nt change. (only copy)
* Destination part always change.

b) Immediate Addressing mode ✓

Immediate data are constant data

data transsfered from a register or memory location are variable data

mov CH, 3AH

c) Direct Data Addressing mode ✓

direct addressing, which applies to a MOV between a memory location and AL, AX or EAX.

MOV [1234H], AX → content of memory location. ✓

* Ax 'deki değeri [1234H] memory location'a kayıyoruz.

d) Register Indirect Addressing

Allows data to be addressed at any memory location through an offset address held in any of the following registers : BP, BX, DI, and SI.

MOV [BX], CL * tabular data located in memory array.

* The data segment is used by default with register indirect addressing or any other mode that uses BX, DI, or SI to address memory.
* If the BP register addresses memory, the stack segment is used by default.

Big endian (big end first)

12	34	56	78

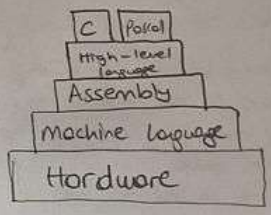
Little endian (little end first)

78	56	34	12

The Microprocessor and Its Architecture

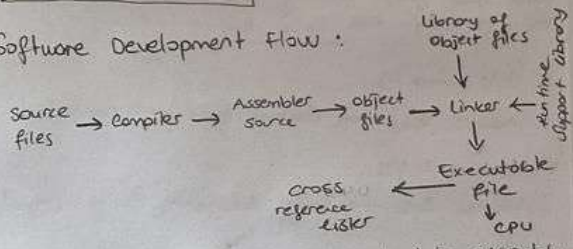
Machine language → lowest-level programming language
↳ only one understood by computers.

Assembly contains some instructions as a machine language, but the instructions and variables have names instead of being just numbers.



Every CPU has its own unique machine language.

Software Development Flow:



Compiler: translates source code into assembly language source code.

Assembler: translates assembly language source files into machine language.

Linker: combines object files created by the assembler into a single executable object module.

Cross-reference lister: uses object files to produce a cross-reference listing showing variable and symbols, their definitions, memory locations and their references in the linked source code.

Intel 8086:

Clock rate of the first edition of 8086 was 5 MHz. Later, 8 and 10 mHz versions were also produced.

A 20 bit external data bus gives a 1MB physical address space.

8086 is divided into two seperate functional unit.

1. Bus Interface Unit (BIU)
2. Execution Unit (EU)

Bus Interface Unit (BIU):

- send out address
- fetches instruction from memory
- read data from ports and memory
- write data to ports and memory

BIU handles all transfers of data and addresses on the buses for the execution unit.

Instruction Queue:

To speed up the program execution BIU fetches as many as 6 instruction bytes ahead of the memory. The pre fetch instruction bytes are held in instruction queue.

When the EU is ready for its next instruction it simply reads the instruction from the queue in the BIU.

Execution Unit (EU):

The EU tells the BIU,
- where to fetch instructions or data from,
- decode instruction and executes instructions.

The EU contains a 16 bit ALU
- add, substract, AND, OR, XOR, increment, decrement, complement, or shift binary numbers

Basic Instruction Cycle:

Fetch → Decode → Execute
↓ okuma ↓ Anlamlandirma ↓ yapma

* PC (program counter): keeps the address of the instruction that is being executed.

The Programming Model

Core 2 considered program visible
- particular registers are used during programming and are specified by the instructions.

Other registers considered to be program invisible.
- not addressable directly during applications programming.

Multipurpose Registers:

- RAX (64), EAX (32), AX (16), AH (8), AL (8) (accumulator)

The accumulator is used for instructions such as multiplications, division, and some of the adjustment instructions

- RBx, EBx, Bx, BH, BL (base index)

The base index registers sometimes hold offset address of a location in the memory system in all versions of the microprocessor

- RCx, ECx, CX, CH, CL (count)

A (count) general purpose register that also holds the count for various instructions

- RDx, EDx, Dx, DH, DL (data)

A (data) general purpose register holds a parts of the result from multiplication or part of dividend before a division

- RBP, EBP or BP (base pointer)

points to a memory location for memory data transfers.

- RDI, EDI or DI (destination index)

often addresses string destination data for the string instructions.

- RSI, ESI or SI (source index)

the (source index) register addresses source string data for the string instructions

Special - Purpose Registers

include RIP, RSP and RFLAGS

- segment registers include cs, DS, ES, SS, FS and GS

- RIP addresses the next instruction in a section of memory.
  - defined as (instruction pointer) a code segment

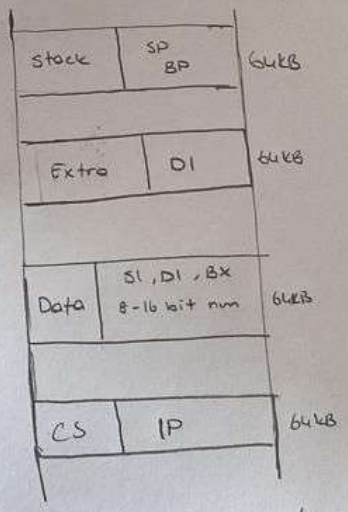
- RSP addresses an area of memory called the stack.
  - the (stack pointer) stores data through this pointer.

- RFLAGS indicate the condition of the microprocessor and control its operation.
Flags never change for any data transfer or program control operation.

- Carry (C) holds the carry after add or borrow after substraction.
- parity (P) is the count of ones in a number expressed as even or odd.
- Auxiliary carry (A) holds the carry (half-carry)
- Zero (Z) shows that the result of an arithmetic or logic operation is zero.
- Sign (S) flags holds the arithmetic sign of the result
- Trap (T) enables trapping through an on-chip debugging feature.
- Interrupt (I) controls operation of the INTR input pin
- Direction (D) selects increment or decrement mode for the DI and/or SI registers
- Overflow (O) occurs when signed numbers are added or subtracted.

Segment Registers:

- CS (code segment) segment holds code used by the mp
- DS (data segment) contains most data used by a program.
- ES (extra) an additional data segment used by some instructions to hold destination data.
- SS (stack) defines the area of memory used for stack.



To find real address : (segment x10H) + offset

- The code segment register defines the start of the code segment.
- The instruction pointer locates the next instruction within the code segment.

Data Movement Instructions

Native binary code microprocessor uses as its instructions to control its operation.

- Instructions vary in length from 1 to 13 bytes.

• 16-bit

8BECH → (binary) 1000 1101 1110 1100 (16 bit instruction)

register / register/memory

OpCode	Mod-REG-R/M	Displacement	Immediate
1-2 bytes	0-1 bytes	0-1 bytes	0-2 bytes

↓

mov Ax, [Bx+100H]

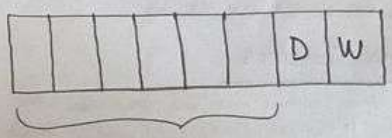
displacement

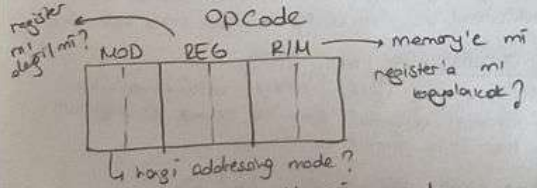
The OpCode

Selects the operation (addition, substraction, etc.)

• First 6 bits of the first byte are the binary opcode

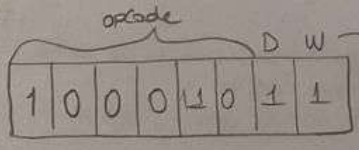
• remaining 2 bits indicate the direction (D) of the data flow, and indicate whether the data are a byte or a word (W)





MOD : defines an addressing mode

Ex:



Opcode : mov

D: Transfer to register (REG)

W: Word

MOD : R/M is a register

REG : BP

R/M : SP

Data Type

BYTE PTR (for byte)

WORD PTR (for word - 2 bytes)

Ex:

mov AL, BYTE PTR [BX] (byte access)

mov CX, WORD PTR [BX] (word access)

mov Instructions

Copies the second operand (source) to the first operand (destination).

Source operand : immediate value, register or memory location

Destination : register or memory location.

* Both operand must be the same size.

* mov inst. cannot be used to set the value of the CS and IP registers.

Operands of mov

```
mov  REG, memory
mov  memory, REG
mov  REG, REG
mov  memory, immediate
mov  REG, immediate
```

REG : AX, BX, CX, DX, AH, AL, BL, BH, CH, CL, DH, DL, DI, SI, BP, SP

memory : [BX], [BX+SI+7], variable, etc.

immediate : 5, -24, 3FH, 10001101b, etc.

Segment Register Operands

SREG : DS, ES, SS, and only as second operand CS.

Ex:

```
mov AX, 0B800h  ; set AX to hexa B800h
mov DS, AX      ; Copy value of AX to DS
mov CL, 'A'     ; set CL to ASCII code of 'A'
mov [BX], CX    ; Copy content of CX to memory
                  at B800:015E
```

Variables

name DB value (Define Byte)
name DW value (Define word)

name: can be any letter or digit combination)
↳ It should start with a letter.
↳ It's possible to declare unnamed variables.

value: can be any numeric value in any supported numbering system.
↳ ? symbol for variables that are not initialized.

ORG Directive

ORG 100h is a assembler directive. It tells assembler that the execute file will be loaded at the offset of 100h (256 bytes)

Directives are never converted to any real machine code.

→ Why executable file is loaded at offset of 100h?
Operating system keeps some data about the program in the first 256 bytes of the CS (code segment) such as command line parameters and etc.

Array

A text string is an example of a byte array, each character is represented as an ASCII code value (0..255)

Ex:

a DB 48h, 65h, 6Ch, 6Fh
b DB 'Hello', 0

If you need to declare a large array you can use DUP operator

c DB 5 DUP(9)
c DB 9,9,9,9,9
d DB 5 DUP (1,2)
d DB 1,2,1,2,1,2,1,2,1,2

You can use DW instead of DB if it required to keep values larger than 255, or smaller than -128.

* DW cannot be used to declare strings.

* The LEA instruction can be used to get the offset address of a variable

Fixed-port addressing
↳ 8-bit I/O port (Follow opcode) → address
Variable-port addressing
(↳ 16-bit port address
port num stored in DX

IN instruction transfers data from an external I/O device into AL, AX, or EAX
OUT instruction data from AL, AX or EAX to an external I/O device

Constants are just like variables, but they exists only until your program compiled.
Constant value cannot be changed.
name EQU <any expression>
K EQU 5

PUSH

Always transfers 2 bytes of data to the stack.
(push all)
PUSHA instruction copies contents of the internal register set, except the segment registers, to the stack.
↳ in the following order:
AX, CX, DX, BX, SP, BP, SI, and DI

PUSHF instruction copies the content of the flag register to the stack

POP

performs the inverse operation of PUSH.
Pop removes data from the stack and places it in a target 16-bit register, segment register, or a 16-bit memory location.

POPF removes a 16-bit number from the stack and places it in the flag register.

POPA removes 16 bytes of data from the stack.
↳ in the order shown
DI, SI, BP, SP, BX, DX, CX, AX

String Data Transfers

Before the string instructions are presented, the operation of the D flag-bit (direction), DI and SI must be understood as they apply to the string instructions.

The Direction Flag (D)

located in flag register, selects auto-increment or auto-decrement operation for the DI and SI registers during the string operations

CLD → clears the D flag (D=0)
STD → set the D flag (D=1)

DI and SI

- DI offset address accesses data in the extra segment for all string instructions that use it.
- SI offset address accesses data by default in the data segment.

LODSB → Load byte at DS:[SI] into AL, update SI
STOSB → Store byte in AL into ES:[DI], update DI
MOVSB → Copy byte at DS:[SI] to ES:[DI] and update DI, SI

IN & OUT instructions perform I/O operation.
Contents of AL, AX or EAX are transferred only between I/O device and microprocessor

Microprocessor and Computer 1

ENIAC → first programmable machine

The first microprocessor by Intel (4004)

John von Neuman. first modern person to develop a system to accept instructions and store them in memory.

4 bit → nibble

8 bit → byte

1 KB 2^{10}

1 MB 2^{20}

1 GB 2^{30}

1 TB 2^{40}

World's first microprocessor the Intel 4004. A 4-bit microprocessor – programmable controller on a chip

A microprocessor is a programmable digital electronic component that incorporates the functions of a central processing unit (CPU) on a single semiconducting integrated circuit (IC)

A microcontroller is a computer-on-a chip.

The only difference between microcontroller and microprocessor is that a microprocessor has free ports - ALU, Control Unit and register, while the microcontroller has additional elements like ROM, RAM, peripheral (timer, I/O ports, etc)

main memory system divide three parts:

- TPA (transiskat program area)
- System area
- XMS (extended memory system)

The TPA holds the DOS (disk operating system) other programs that control the computer system. DOS memory map shows how areas of TPA are used for system programs, data and drivers.

The system area contains programs on read only (ROM) or flash memory, and areas of read/write (RAM) memory for data storage.

I/O devices allow the microprocessor to communicate with the outside world.

Microprocessor and Computer 2

Control memory and I/O through connections called bus.

★ Microprocessor performs three main tasks:

- data transfer between itself and the memory or I/O systems
- simple arithmetic and logic operations
- program flow via simple decisions.

Buses transfer address, data and, control information between microprocessor, memory ad, I/O.

Three buses exist: address (20 bit), data (8 bit) and control.

? The mp reads a memory location by sending the memory an address through the address bus. Next, it sends a memory read control signal to cause the memory to read data. Data read from memory are passed to the mp through the data bus.

? I/O → CPU

Address Bus → Control Signal → Data Bus

? CPU → I/O

Address Bus → Data Bus → Control Signal

Signals:

$\overline{MRDC}$: memory read control

$\overline{MWDC}$: memory write control

$\overline{IORC}$: I/O read control

$\overline{IOWC}$: I/O write control

- Standart ASCII code is 7 bit code.
- Unicode system: stores each character as 16 bit data

Packed form: packed BCD data stored as two digits per byte.

Unpacked form: unpacked BCD data stored as one digit per byte.

Decimal	packed	unpacked
12	0001 0010	0000 0001 0000 0010

Unsigned integer: 0 - 255

Signed integer: -128, 127

? Little endian: The LSB always stored in the lowest-numbered memory locatio

? Big endian: Numbers are stored with the lowe location containing the most sign. data.